Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное учреждение высшего образования

«Томский государственный университет систем управления и радиоэлектроники»

Факультет систем управления (ФСУ)

Кафедра автоматизированных систем управления (АСУ)

Отчет по лабораторной работе №2

по дисциплине «Современные средства программирования»

Обучающийся гр. 435-М1

_____Андреев Д.П.

«_____»_____2025 г.

Проверил: доцент каф. АСУ, к.т.н

_____Алфёров С.М,

«___»_____2025 г.

Томск 2025

1. Цель работы.

Получить навыки распараллеливания задач и написания параллельных потоков для неразделяемых ресурсов.

2. Индивидуальное задание

Дана прямоугольная матрица заданного типа. “Разделить” матрицу на подматрицы произвольным способом. Каждый поток должен обрабатывать отдельную подматрицу так, чтобы в целом решалась задача, соответствующая варианту. Искомое значение должно быть оформлено как общий ресурс для всех потоков, таким образом он будет не разделяемым. Разрешить проблему неразделяемого ресурса при помощи симафоров или мютексов.

Проверить правильность решенной задачи вычислив искомое значение в основном потоке. Запустить программу при разном количестве столбцов и строк матрицы.

Вариант 2: Тип матрицы: вещественная.

Искомое значение: количество элементов входящих в заданный диапазон

3. Решение

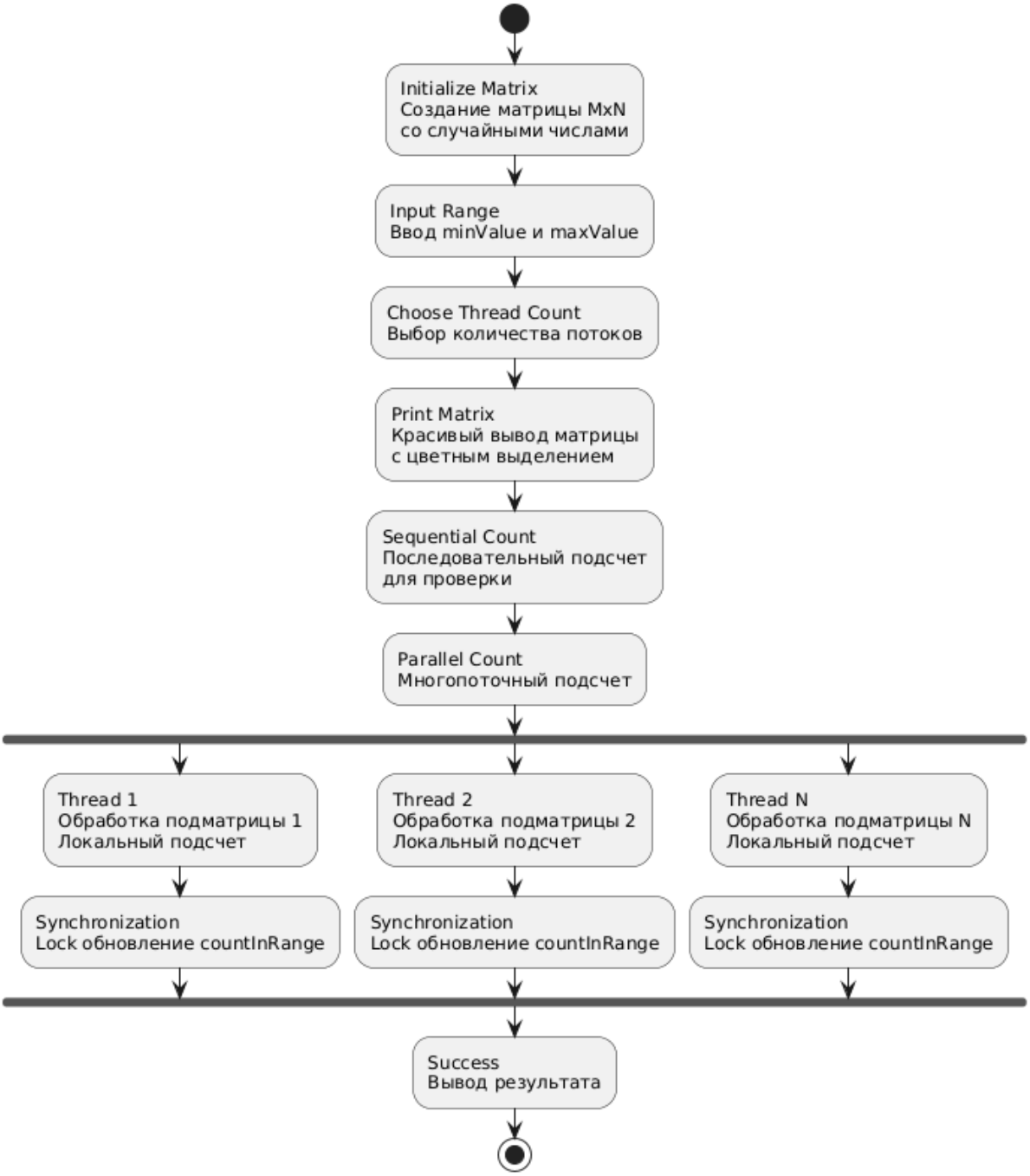

Рисунок 3.1 – UML-диаграмма деятельности

Листинг программы:

```
using System;
```

```
using System.Threading;
using System.Diagnostics;

namespace ССП_2
{
    class Program
    {
        // Общий ресурс - счетчик элементов в диапазоне
        private static int countInRange = 0;
        private static readonly object lockObject = new object();

        // Параметры задачи
        private static double minValue, maxValue;
        private static double[,] matrix;

        static double[,] CreateMatrix()
        {
            Console.Write("Введите количество столбцов: ");
            int columns = Convert.ToInt32(Console.ReadLine());
            Console.Write("Введите количество строк: ");
            int lines = Convert.ToInt32(Console.ReadLine());
            matrix = new double[lines, columns];
            Random r = new Random();
            for (int i = 0; i < lines; i++)
            {
                for (int j = 0; j < columns; j++)
                {
                    matrix[i, j] = r.Next(-10, 10);
                }
            }
            return matrix;
        }//Создание матрицы
        static void PrintMatrix()
        {
            for (int i = 0; i < matrix.GetLength(0); i++)
            {
```

```
        for (int j = 0; j < matrix.GetLength(1); j++)
        {
            Console.Write($"{matrix[i, j],8:F2}");
        }
        Console.WriteLine("");
    }
}//Вывод матрицы
static void InputRange()//Ввод диапозона
{
    Console.Write("Введите минимальное значение диапазона: ");
    minValue = Convert.ToDouble(Console.ReadLine());
    Console.Write("Введите максимальное значение диапазона: ");
    maxValue = Convert.ToDouble(Console.ReadLine());

    if (minValue > maxValue)
    {
        Console.WriteLine("Минимальное значение больше максимального! Меняем местами.");
        double temp = minValue;
        minValue = maxValue;
        maxValue = temp;
    }

    Console.WriteLine($"Диапазон для поиска: [{minValue}, {maxValue}]");
}
static int CountInRangeSequential()
{
    int count = 0;
    int rows = matrix.GetLength(0);
    int cols = matrix.GetLength(1);

    for (int i = 0; i < rows; i++)
    {
        for (int j = 0; j < cols; j++)
        {
            if (matrix[i, j] >= minValue && matrix[i, j] <= maxValue)
            {
```

```
                count++;
            }
        }
    }
    return count;
}//Последовательный подсчет
static void CountInRangeParallel(int threadCount)
{
    countInRange = 0;

    int rows = matrix.GetLength(0);
    int cols = matrix.GetLength(1);

    // Создаем и запускаем потоки
    Thread[] threads = new Thread[threadCount];

    for (int i = 0; i < threadCount; i++)
    {
        int threadIndex = i;
        threads[i] = new Thread(() => ProcessSubmatrix(threadIndex, threadCount, rows, cols));
        threads[i].Start();
    }

    // Ожидаем завершения всех потоков
    foreach (Thread thread in threads)
    {
        thread.Join();
    }

}//Организация многопоточной обработки
static void ProcessSubmatrix(int threadIndex, int totalThreads, int totalRows, int totalCols)
{
    // Разделяем матрицу по строкам
    int rowsPerThread = totalRows / totalThreads;
    int startRow = threadIndex * rowsPerThread;
    int endRow = (threadIndex == totalThreads - 1) ? totalRows : startRow + rowsPerThread;
```

```
    int localCount = 0;

    // Подсчитываем элементы в диапазоне для своей подматрицы
    for (int i = startRow; i < endRow; i++)
    {
        for (int j = 0; j < totalCols; j++)
        {
            if (matrix[i, j] >= minValue && matrix[i, j] <= maxValue)
            {
                localCount++;
            }
        }
    }

    // Безопасно добавляем локальный счетчик к общему
    lock (lockObject)
    {
        countInRange += localCount;
    }

    Console.WriteLine($"Поток {threadIndex}: найдено {localCount} элементов");
}

static void Main(string[] args)
{
    // Инициализация матрицы
    CreateMatrix();

    // Ввод диапазона
    InputRange();

    // Вывод исходной матрицы
    PrintMatrix();

    DateTime startTime = DateTime.Now;
```

```
        // Подсчет в основном потоке для проверки
        int sequentialCount = CountInRangeSequential();
        DateTime endTime = DateTime.Now;
        Console.WriteLine($"\nРезультат последовательного подсчета: {sequentialCount}, Время: {(endTime-startTime).TotalMilliseconds} ms");



        // Многопоточный подсчет
        startTime = DateTime.Now;
        int threadCount = Environment.ProcessorCount;
        Console.WriteLine($"Количество логических процессоров: {threadCount}");
        Math.Min(threadCount, matrix.GetLength(0));
        CountInRangeParallel(threadCount);
        endTime = DateTime.Now;
        Console.WriteLine($"Результат параллельного подсчета: {countInRange}, Время: {(endTime - startTime).TotalMilliseconds} ms");


        // Проверка корректности
        if (sequentialCount == countInRange)
        {
            Console.WriteLine("Результаты совпадают! Программа работает корректно.");
        }
        else
        {
            Console.WriteLine("Результаты не совпадают! Возникла ошибка в синхронизации.");
        }
        Console.ReadKey();


    }
  }
}
```

4. Результаты работы

```
C:\Users\Lion120454#2\source\repos\ССП_2\ССП_2\bin\Debug\ССП_2.exe
Введите количество столбцов: 6
Введите количество строк: 4
Введите минимальное значение диапазона: 2
Введите максимальное значение диапазона: 7
```

Рисунок 4.1 – Ввод параметров

```
Введите количество столбцов: 6
Введите количество строк: 4
Введите минимальное значение диапазона: 2
Введите максимальное значение диапазона: 7
Диапазон для поиска: [2, 7]
    9,00   -1,00    6,00    0,00    0,00    6,00
    5,00   -7,00    5,00    6,00   -1,00    3,00
    6,00    2,00   -8,00   -5,00  -10,00  -10,00
    7,00    3,00    0,00    4,00    5,00    4,00

Результат последовательного подсчета: 13, Время: 3,0073 ms
Количество логических процессоров: 8
Поток 0: найдено 0 элементов
Поток 1: найдено 0 элементов
Поток 2: найдено 0 элементов
Поток 4: найдено 0 элементов
Поток 3: найдено 0 элементов
Поток 5: найдено 0 элементов
Поток 6: найдено 0 элементов
Поток 7: найдено 13 элементов
Результат параллельного подсчета: 13, Время: 185,3071 ms
Результаты совпадают! Программа работает корректно.
```

Рисунок 4.2 –Результат программы

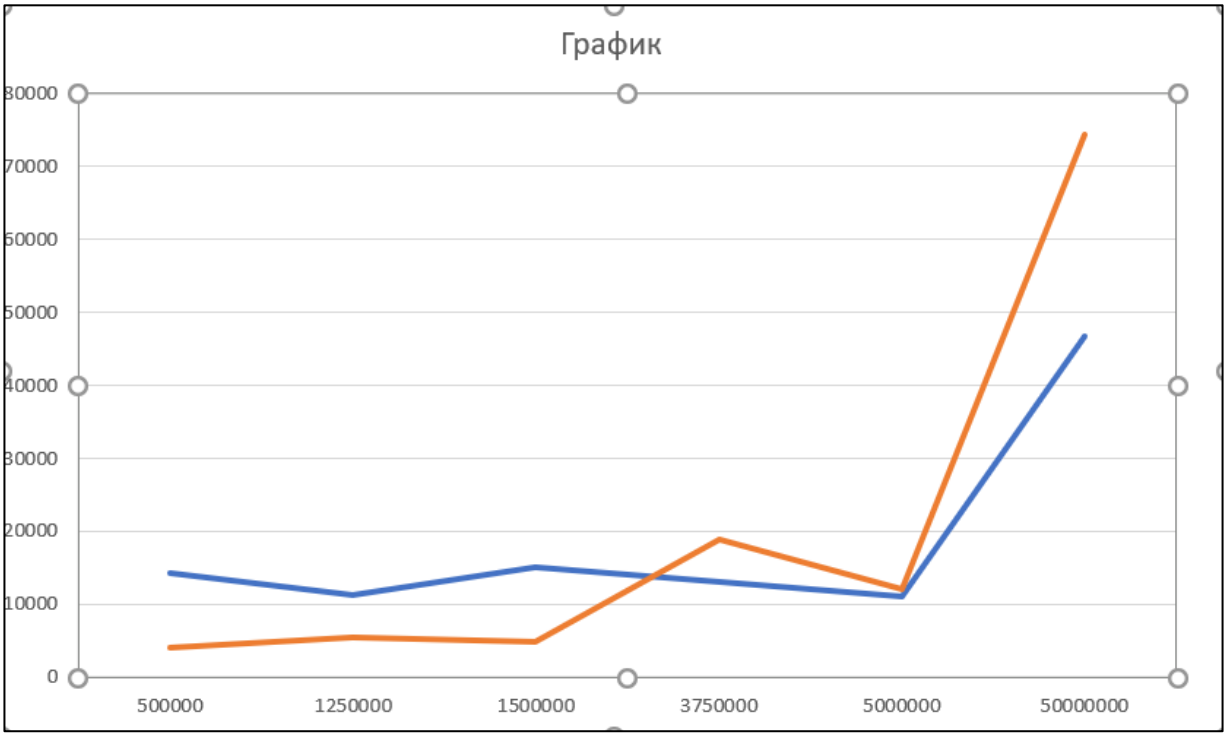

Рисунок 4.3 –График сравнения

5. Вывод

В результате выполнения лабораторной работы я получил навык распараллеливания задач и написания параллельных потоков для неразделяемых ресурсов.